Synchronous FIFO Documentation

Problem Explanation

A FIFO (First-In-First-Out) memory stores data such that the first data written is the first one to be read. This project implements a synchronous FIFO using a RAM-based architecture in Verilog with control signals for read, write, full, and empty conditions.

Approach / Logic Used

The FIFO uses a 4x4 memory (RAM) with write and read pointers. Data is written when wr_en is high and FIFO is not full. Data is read when rd_en is high and FIFO is not empty. Pointers increment on each valid operation.

✅ **Outputs:**

Signal	Size	Meaning
data_out	4-bit	Output data
full	1-bit	FIFO is full
empty	1-bit	FIFO is empty

Block Diagram

Components:
- Input signals (clk, rst, wr_en, rd_en, data_in)
- RAM (4x4)
- Write pointer and Read pointer
- Control logic (full and empty flags)
- Output (data_out)

Code (Synchronous FIFO)

```
module sync_fifo (
   input clk,
   input rst,
   input wr_en,
```

```
    input rd_en,
    input [3:0] data_in,
    output reg [3:0] data_out,
    output reg full,
    output reg empty
);

    reg [3:0] mem [0:3];
    reg [1:0] wr_ptr, rd_ptr;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            wr_ptr <= 0;
            rd_ptr <= 0;
            full   <= 0;
            empty  <= 1;
        end else begin

            if (wr_en && !full) begin
                mem[wr_ptr] <= data_in;
                wr_ptr <= wr_ptr + 1;
            end

            if (rd_en && !empty) begin
                data_out <= mem[rd_ptr];
                rd_ptr <= rd_ptr + 1;
            end

            if (wr_ptr == rd_ptr)
                empty <= 1;
            else
                empty <= 0;

            if ((wr_ptr + 1) == rd_ptr)
                full <= 1;
            else
                full <= 0;

        end
    end

endmodule
```

Testbench

```
module tb_sync_fifo;

 reg clk, rst;
 reg wr_en, rd_en;
 reg [3:0] data_in;

 wire [3:0] data_out;
 wire full, empty;

 integer i;

 sync_fifo uut (
  .clk(clk),
  .rst(rst),
  .wr_en(wr_en),
  .rd_en(rd_en),
  .data_in(data_in),
  .data_out(data_out),
  .full(full),
  .empty(empty)
 );

 always #5 clk = ~clk;

 initial begin
  clk = 0;
  rst = 1;
  wr_en = 0;
  rd_en = 0;

  #10 rst = 0;

  for (i = 0; i < 4; i = i + 1) begin
   @(posedge clk);
   wr_en = 1;
   data_in = i;
  end

  @(posedge clk);
  wr_en = 0;

  for (i = 0; i < 4; i = i + 1) begin
```

```
    @(posedge clk);
    rd_en = 1;
    @(posedge clk);
  end

  rd_en = 0;
  #20 $finish;
 end

endmodule
```

Code Explanation

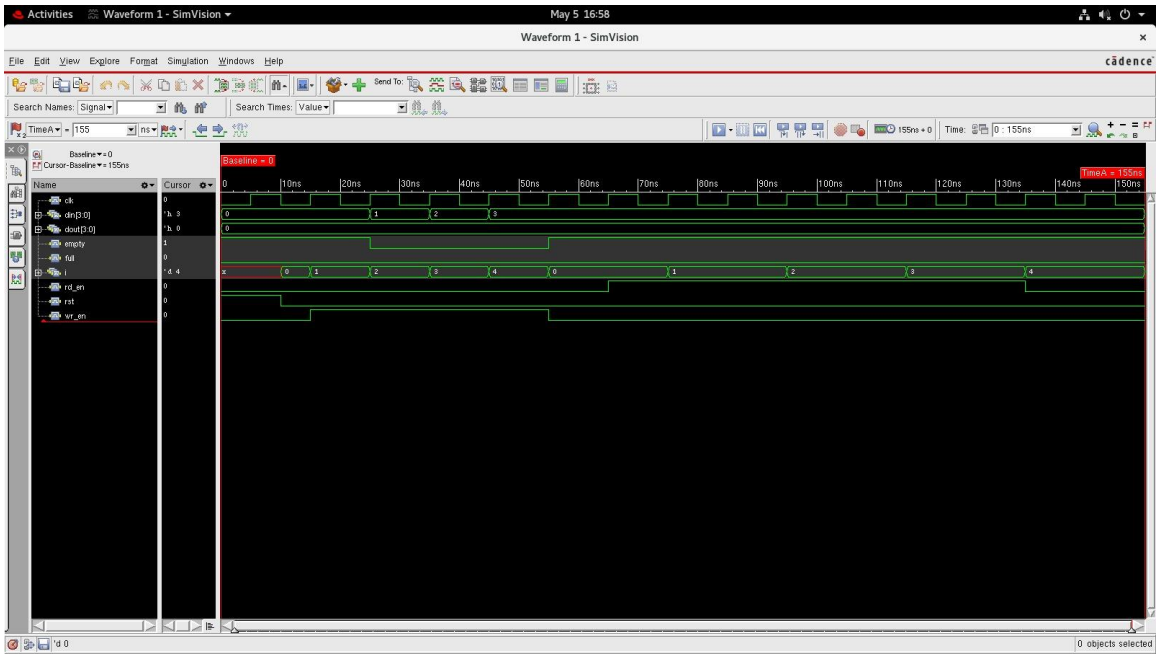
The design uses a RAM array to store data. Write pointer tracks where to write and read pointer tracks where to read. Flags are generated based on pointer positions.

Testbench Details

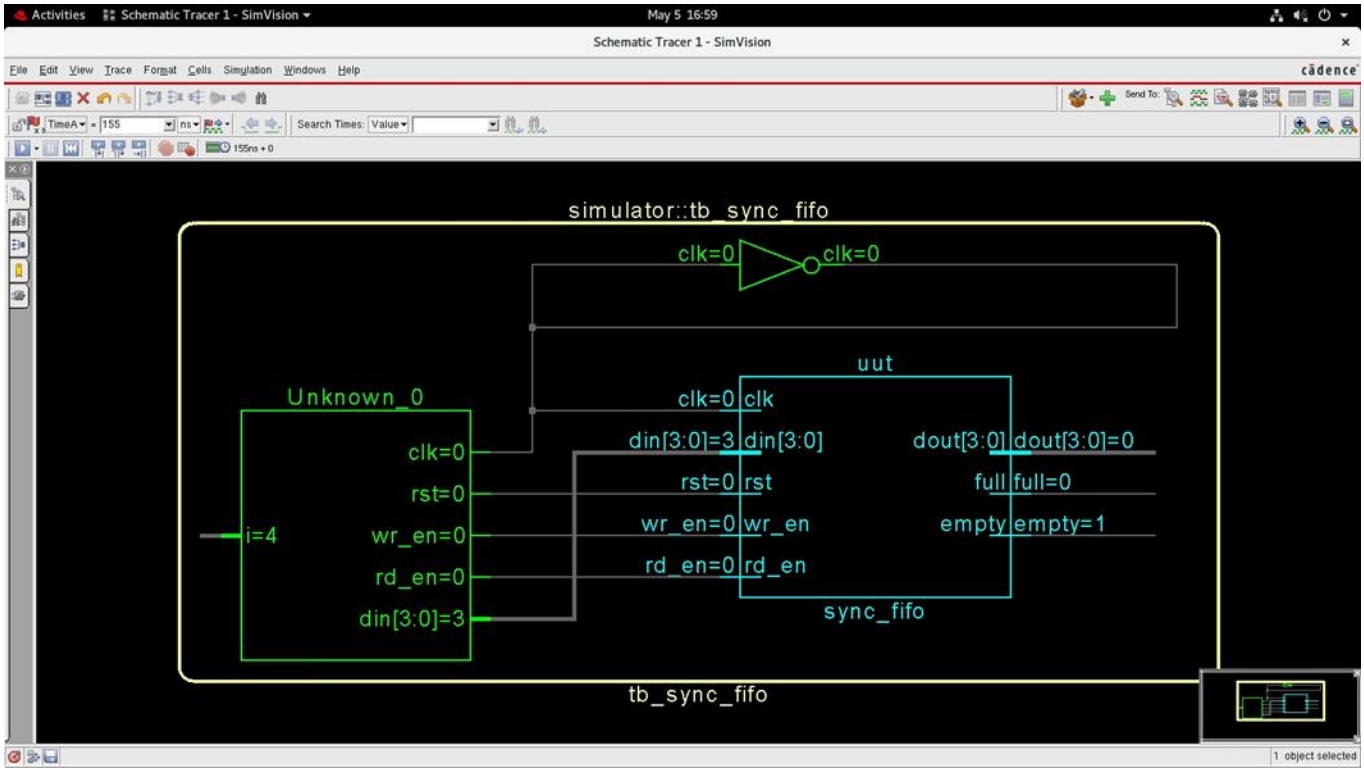
The testbench generates a clock, applies reset, writes 4 values into FIFO, then reads them back while observing outputs.

Waveform Analysis

Data is written sequentially and read in the same order. Full becomes high when FIFO is filled, and empty becomes high after all data is read.



Schematic diagram



Challenges Faced

Handling correct full/empty logic, pointer wrap-around, and avoiding invalid read/write conditions.